

# Capítulo 5

# Sistema de Ficheros

Ficheros

Directorios

Implementación del sistema de ficheros

# Ficheros (1)

## Almacenamiento de información a largo plazo

1. Almacenar grandes cantidades de datos (más de lo que cabe en el espacio virtual de direccionamiento)
2. La información debe vivir más allá de la terminación del proceso que la utiliza (persistencia)
3. Múltiples procesos quieren acceder a la misma información concurrentemente

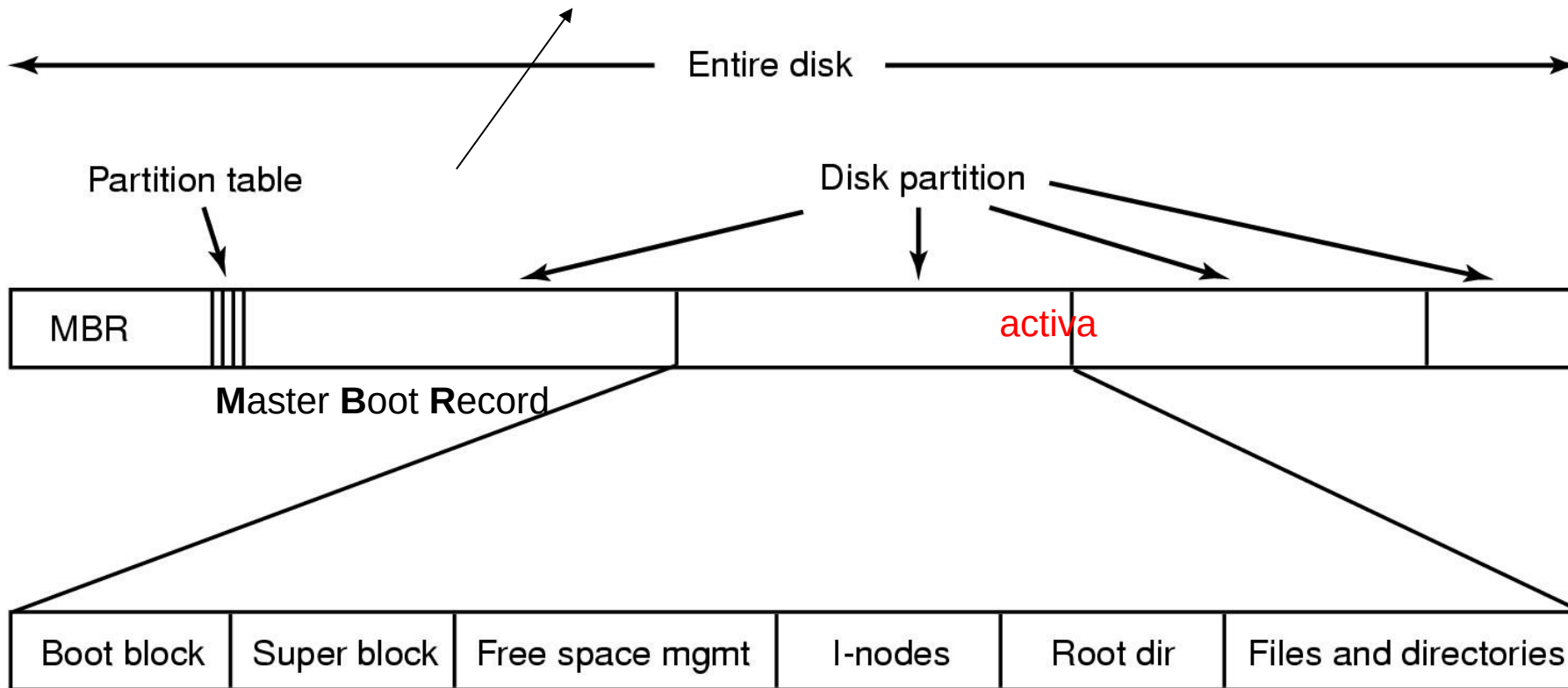
Solución: almacenar la información en unidades llamadas **Ficheros**

Sistema Operativo: **Gestor de Ficheros (Sistema de Ficheros)**

Veremos: la perspectiva del usuario y la del SO

# Implementación del sistema de ficheros. (Estructura) (1)

Direcciones de comienzo y fin



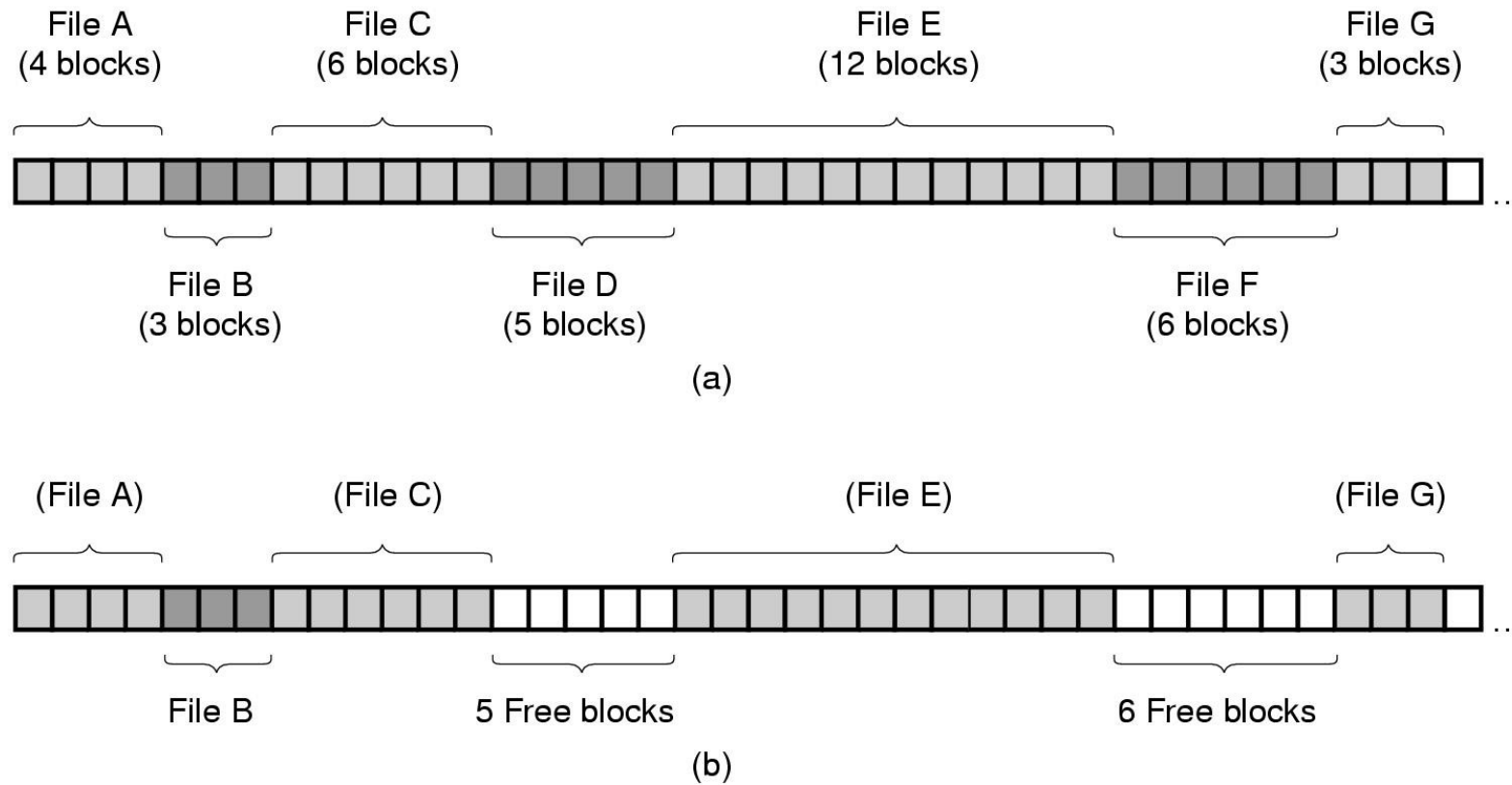
PowerOn: BIOS lee y ejecuta MBR; MBR localiza partición activa, lee y ejecuta el boot block

# Implementación del sistema de ficheros.

## (implementación de ficheros) (2)

¿En qué bloques del disco está el contenido de un fichero?

### Asignación contigua



# Implementación del sistema de ficheros. (implementación de ficheros) (3)

¿En qué bloques del disco está el contenido de un fichero?

## Asignación contigua (ventajas)

- Fácil implementar
- ¿Dónde están los bloques de un fichero?
  - dirección del primer bloque
  - número de bloques
- buen rendimiento en lectura: solo 1 seek 1 rotación

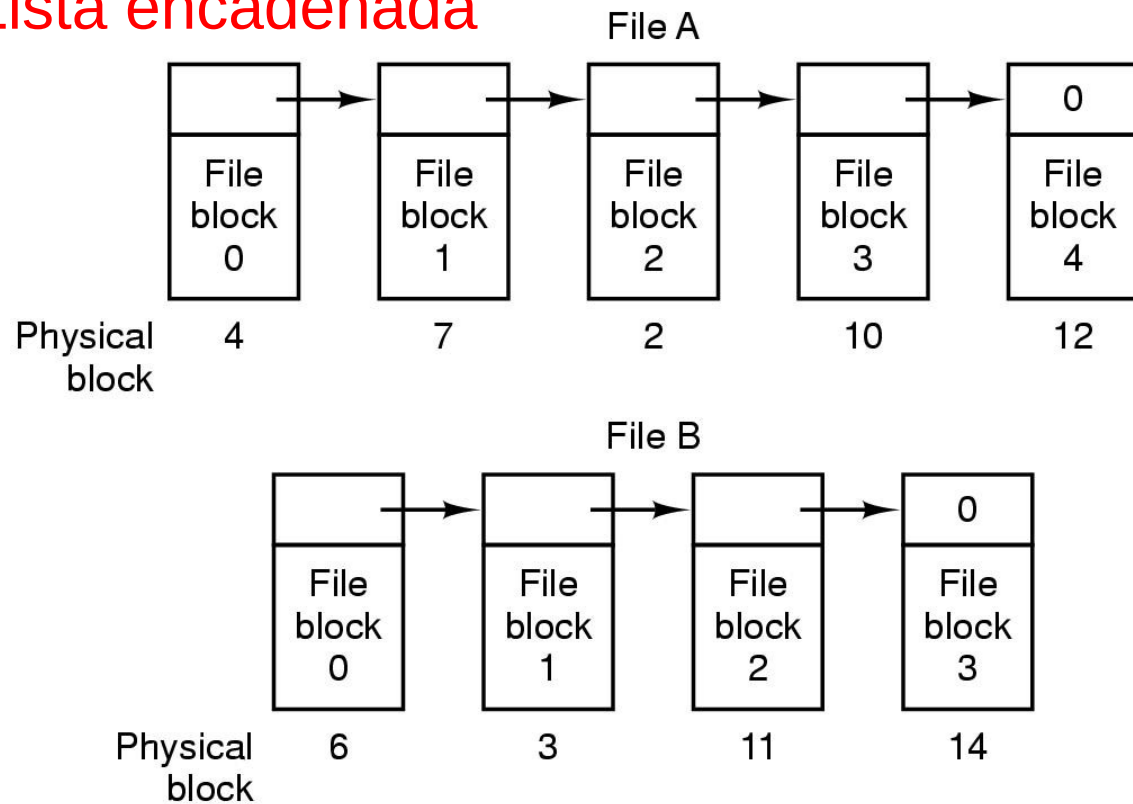
## Asignación contigua (desventajas)

- Encontrar un hueco lo bastante grande
- Hay que conocer el tamaño final del fichero
- La compactación es cara
- Adecuado para CDs

# Implementación del sistema de ficheros. (implementación de ficheros) (4)

¿En qué bloques del disco está el contenido de un fichero?

## Lista encadenada



# Implementación del sistema de ficheros. (implementación de ficheros) (5)

¿En qué bloques del disco está el contenido de un fichero?

## Lista encadenada

- No fragmentación externa, SI fragmentación interna
- En la entrada al directorio: dirección del primer bloque
- Mal para acceso aleatorio
- La cantidad de bytes en un bloque no es potencia de 2
  - los programas R/W en potencias de 2
  - hay que acceder a dos bloques

# Implementación del sistema de ficheros. (implementación de ficheros) (6)

¿En qué bloques del disco está el contenido de un fichero?

Lista encadenada utilizando una tabla en memoria

## File Allocation Table

| Physical block |    |                      |
|----------------|----|----------------------|
| 0              |    |                      |
| 1              |    |                      |
| 2              | 10 |                      |
| 3              | 11 |                      |
| 4              | 7  | ← File A starts here |
| 5              |    |                      |
| 6              | 3  | ← File B starts here |
| 7              | 2  |                      |
| 8              |    |                      |
| 9              |    |                      |
| 10             | 12 |                      |
| 11             | 14 |                      |
| 12             | -1 |                      |
| 13             |    |                      |
| 14             | -1 |                      |
| 15             |    | ← Unused block       |

# Implementación del sistema de ficheros. (implementación de ficheros) (7)

¿En qué bloques del disco está el contenido de un fichero?

**Lista encadenada utilizando una tabla en memoria (FAT)**

Ventajas:

- Todo el bloque para datos
- Acceso aleatorio mejorado
- Entrada al directorio: primer bloque

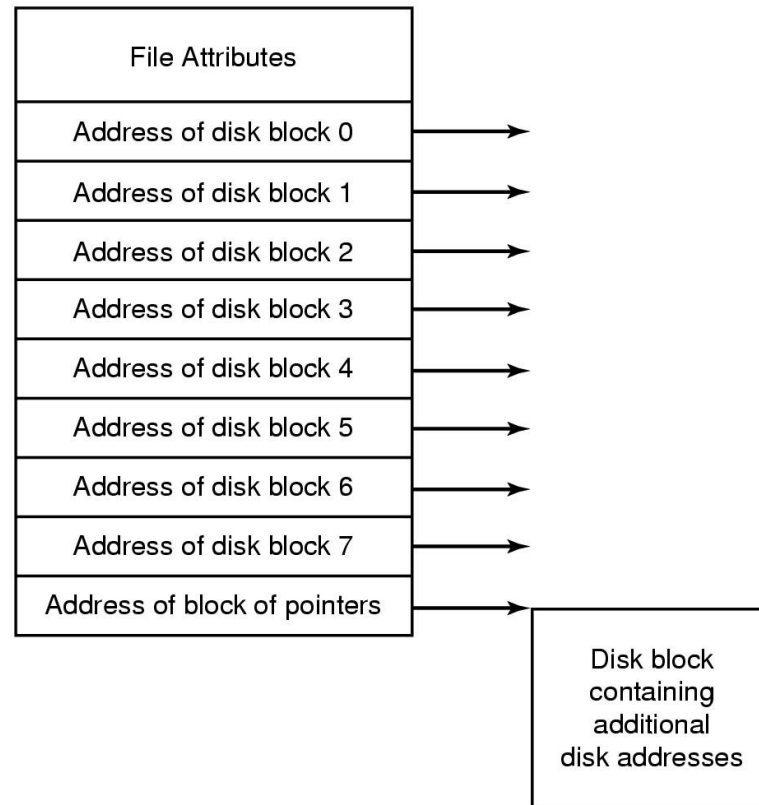
Desventajas:

- Toda la tabla debe estar en memoria.  
disco: 20Gb; bloque: 1K; nº entradas: 20 millones  
tamaño de la entrada: 3 bytes; tamaño de la tabla: 60 Mb (en memoria)

# Implementación del sistema de ficheros. (implementación de ficheros) (8)

¿En qué bloques del disco está el contenido de un fichero?

I-node



# Implementación del sistema de ficheros. (implementación de ficheros) (9)

¿En qué bloques del disco está el contenido de un fichero?

## I-node

Ventajas:

- Solo en memoria el i-node del fichero abierto
- Tabla de i-nodes residentes en memoria (para todos los procesos)
- Tabla de ficheros abiertos (por proceso)
- Cada proceso tiene un máximo de ficheros abiertos

Desventajas:

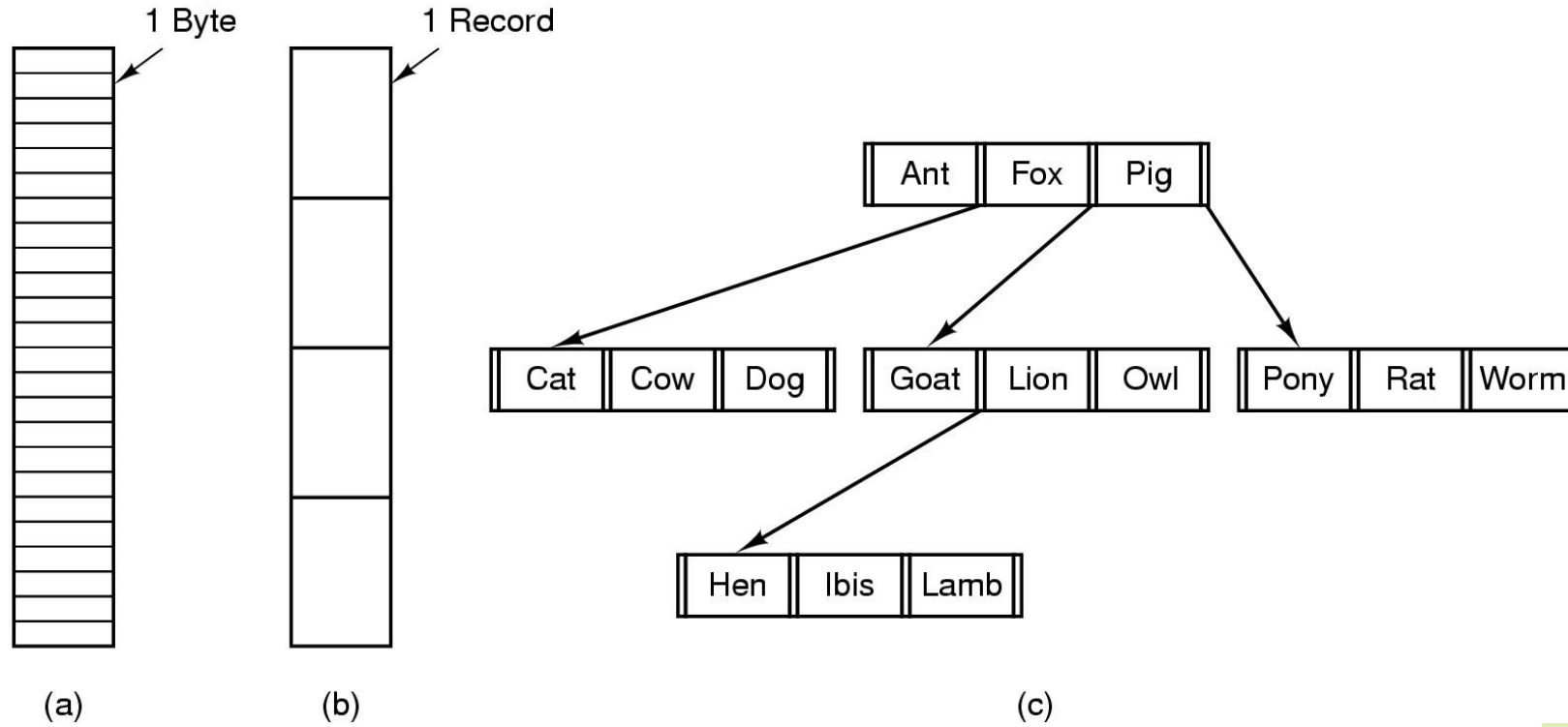
- Los punteros indirectos aumentan el tiempo de búsqueda de bloques

# Ficheros. (Nombrado) (2)

| Extension | Meaning                                           |
|-----------|---------------------------------------------------|
| file.bak  | Backup file                                       |
| file.c    | C source program                                  |
| file.gif  | Compuserve Graphical Interchange Format image     |
| file.hlp  | Help file                                         |
| file.html | World Wide Web HyperText Markup Language document |
| file.jpg  | Still picture encoded with the JPEG standard      |
| file.mp3  | Music encoded in MPEG layer 3 audio format        |
| file.mpg  | Movie encoded with the MPEG standard              |
| file.o    | Object file (compiler output, not yet linked)     |
| file.pdf  | Portable Document Format file                     |
| file.ps   | PostScript file                                   |
| file.tex  | Input for the TEX formatting program              |
| file.txt  | General text file                                 |
| file.zip  | Compressed archive                                |

Extensiones típicas de un fichero

# Ficheros. (Estructura) (3)



- 3 tipos de estructuración
  - (a) Secuencia de bytes (Unix, Windows): interpreta el usuario
  - (b) Secuencia de registros: la unidad de R/W es el registro
  - (c) Árbol, cada nodo contiene una clave

# Ficheros. (Tipos) (4)

Varios tipos de ficheros.

**Windows:** regulares, directorios

**Unix:** regulares, directorios, especiales, pseudoficheros

**Regulares:** información del usuario, del tipo

binarios: ejecutables, archivos

ASCII: líneas de texto acabadas en: cr o

lf (UNIX), o crlf (MSDOS)

**Directorios:** mantienen la estructura del sistema de ficheros

**Especiales de caracteres:** impresora, terminal, network

**Especiales de bloques:** discos, disquettes

# Ficheros. (Acceso) (5)

- Acceso secuencial
  - Lee todos los bytes/registros desde el principio
  - No puede saltar, solo rebobinar
  - Adecuado con cintas magnéticas
- Acceso aleatorio
  - Se leen bytes/registros en cualquier orden
  - Esencial para bases de datos
  - En la lectura ...
    - Se da la posición desde donde leer, o ...
    - Seek, read

# Ficheros. (Atributos) (6)

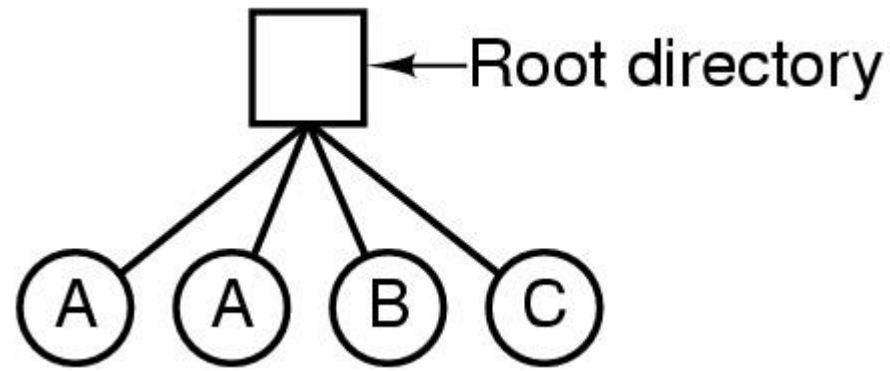
| Attribute           | Meaning                                               |
|---------------------|-------------------------------------------------------|
| Protection          | Who can access the file and in what way               |
| Password            | Password needed to access the file                    |
| Creator             | ID of the person who created the file                 |
| Owner               | Current owner                                         |
| Read-only flag      | 0 for read/write; 1 for read only                     |
| Hidden flag         | 0 for normal; 1 for do not display in listings        |
| System flag         | 0 for normal files; 1 for system file                 |
| Archive flag        | 0 for has been backed up; 1 for needs to be backed up |
| ASCII/binary flag   | 0 for ASCII file; 1 for binary file                   |
| Random access flag  | 0 for sequential access only; 1 for random access     |
| Temporary flag      | 0 for normal; 1 for delete file on process exit       |
| Lock flags          | 0 for unlocked; nonzero for locked                    |
| Record length       | Number of bytes in a record                           |
| Key position        | Offset of the key within each record                  |
| Key length          | Number of bytes in the key field                      |
| Creation time       | Date and time the file was created                    |
| Time of last access | Date and time the file was last accessed              |
| Time of last change | Date and time the file has last changed               |
| Current size        | Number of bytes in the file                           |
| Maximum size        | Number of bytes the file may grow to                  |

# Ficheros. (Operaciones) (7)

1. Create
2. Delete
3. Open
4. Close
5. Read
6. Write
7. Append
8. Seek
9. Get attributes
10. Set Attributes
11. Rename

# Directorios. (de un nivel) (1)

Directorios (folders, carpetas): estructuran el sistema de ficheros.



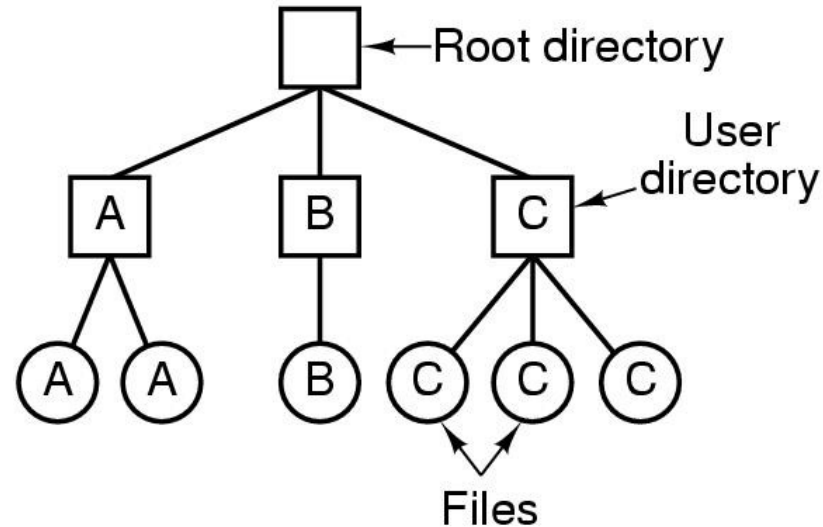
Es simple, fácil de implementar.

Colisión de nombres (solo un espacio de nombrado)

Tampoco útil con un solo usuario (miles de ficheros)

# Directorios. (de dos niveles) (2)

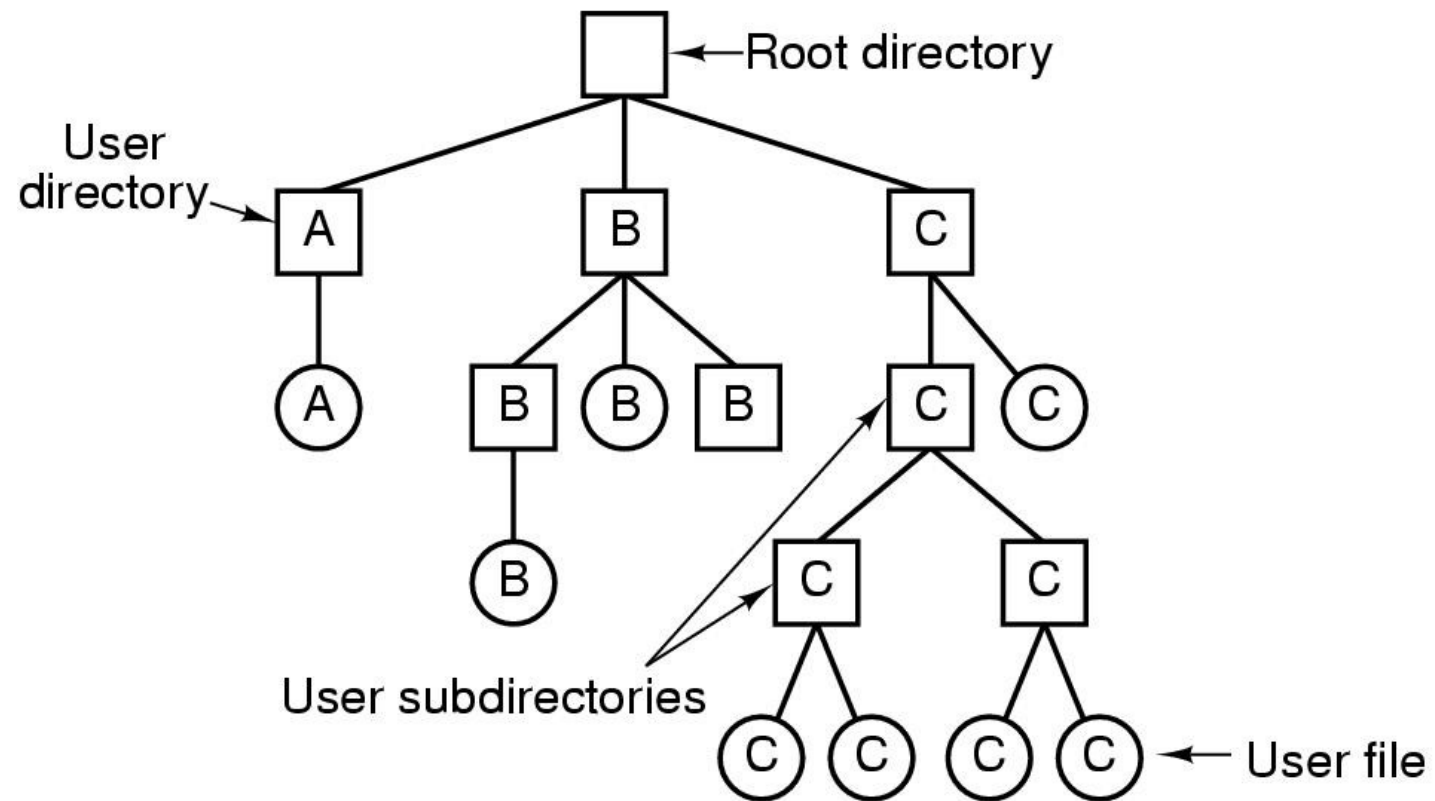
Cada usuario tiene su propio directorio



Identificación del usuario

Acceso a ficheros que no son del usuario (p.e. del sistema)

# Directorios. (jerárquico) (3)



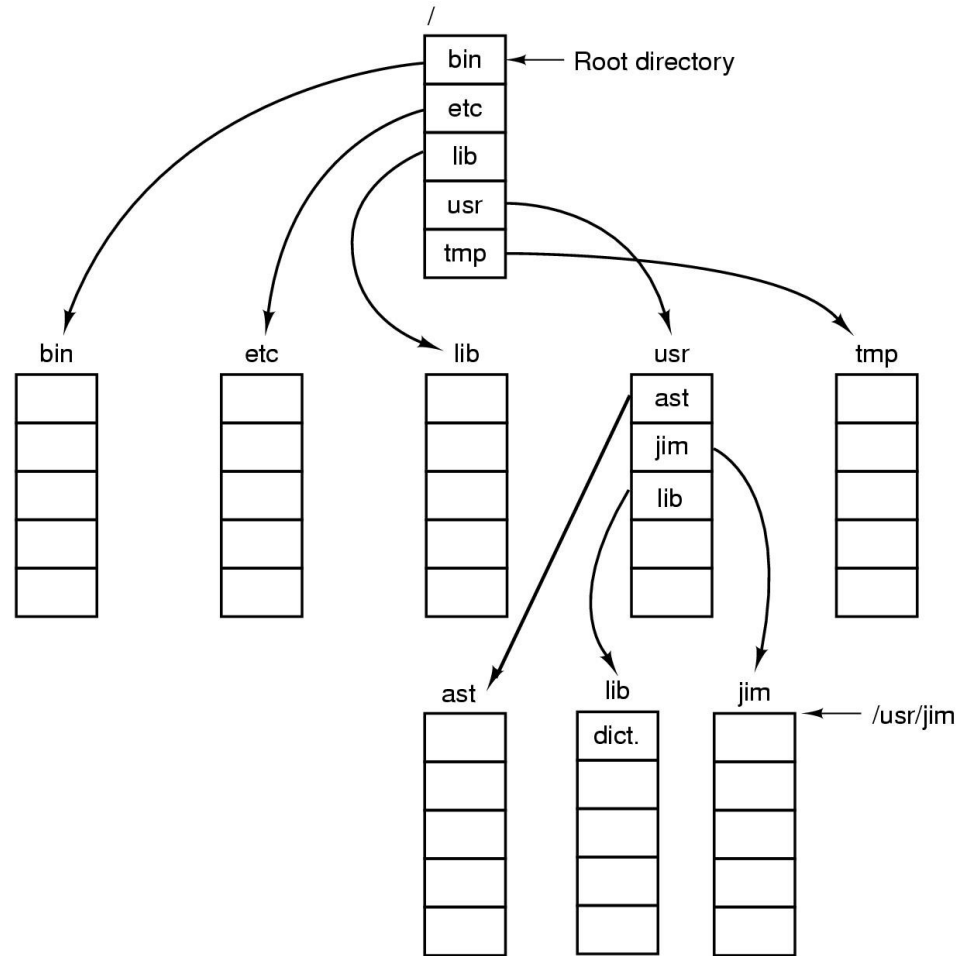
# Directorios. (Path Names) (4)

Nombre absoluto

Nombre relativo

Directorio de trabajo

Entradas: . ..



## Directorios. (Operaciones) (5)

1. Create
2. Delete
3. Opendir
4. Closedir
5. Readdir
6. Rename
7. Link
8. Unlink

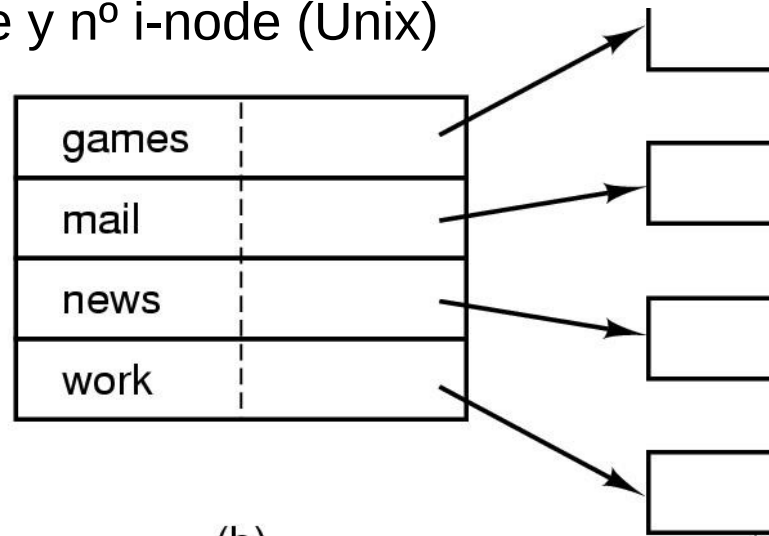
# Implementación del sistema de ficheros. (implementación de directorios) (10)

Cada entrada al directorio mantiene los bloques de disco del fichero y sus atributos

- (a) Entrada: nombre, atributos y bloques de disco (MSDOS, Windows)
- (b) Entrada: nombre y nº i-node (Unix)

|       |            |
|-------|------------|
| games | attributes |
| mail  | attributes |
| news  | attributes |
| work  | attributes |

(a)



(b)

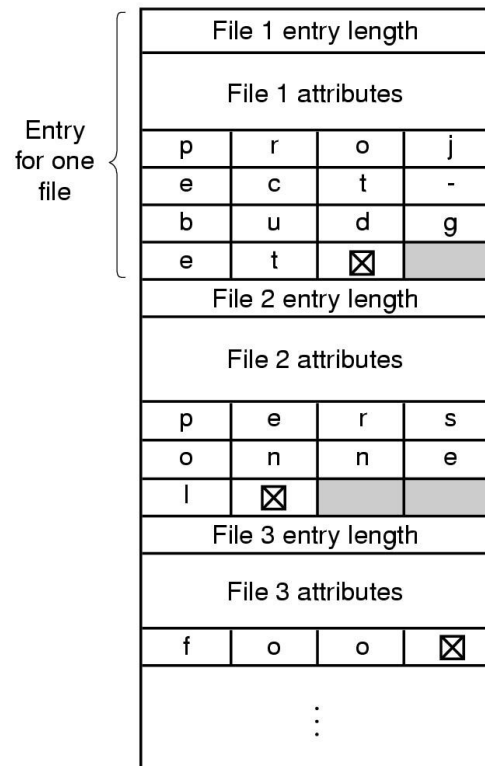
Data structure  
containing the  
attributes

# Implementación del sistema de ficheros. (implementación de directorios) (11)

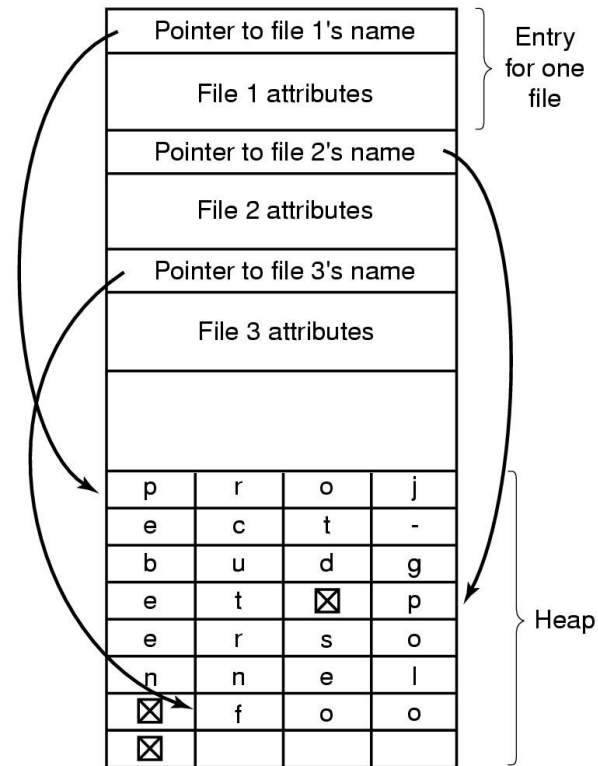
Cada entrada al directorio mantiene el nombre del fichero: longitud fija

MSDOS: 8+3;      Unix V.7: de 1 a 14

¿Longitud del nombre variable y más grande?



(a)



(b)

# Implementación del sistema de ficheros. (implementación de directorios) (12)

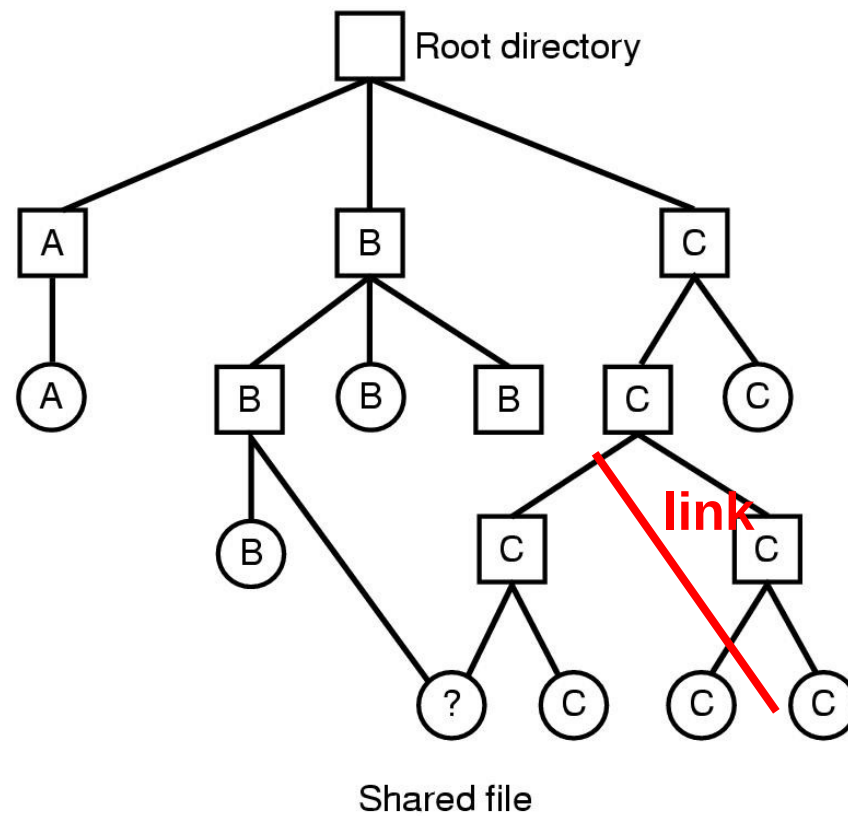
- (a) Entrada al directorio de longitud variable
  - Cuando se elimina queda un hueco de tamaño variable
- (b) Entrada al directorio de longitud fija
  - los nombres de los ficheros al final del directorio
  - ventaja: no huecos de longitud variable

Búsqueda lineal en el directorio lenta (si muchas entradas)

Mejor: tabla hash en cada directorio

Opción: memoria caché

# Implementación del sistema de archivos. (ficheros compartidos) (13)



# Implementación del sistema de ficheros. (ficheros compartidos) (14)

Problema: si las entradas contienen las direcciones de disco

La entrada de B contiene una copia de la entrada de C

Si B o C hacen crecer el fichero, solo se modifica una entrada

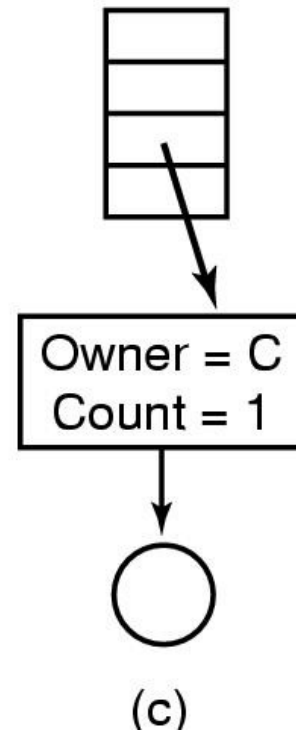
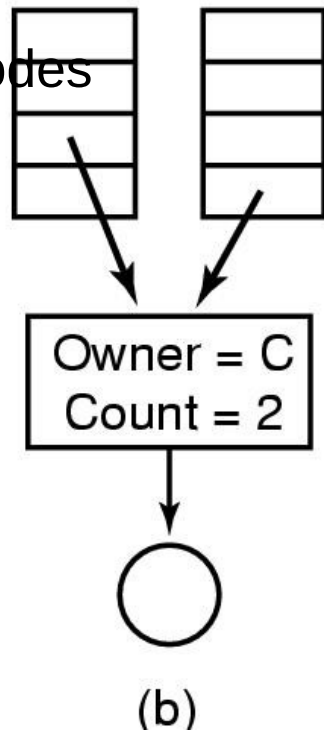
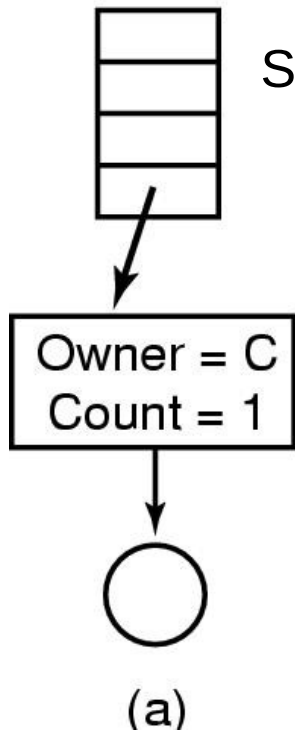
C's directory

B's directory

C's directory

B's directory

Solución 1: i-nodes



# Implementación del sistema de ficheros. (ficheros compartidos) (15)

Solución 2: enlaces simbólicos (ficheros tipo link)

- No presentan el problema anterior: si el propietario elimina el fichero, el contenido del fichero link indica un fichero que no existe.
- Además, un enlace simbólico puede enlazar ficheros en otros nodos.
- Presenta otros problemas:
  - sobrecarga por tener el nombre y no el i-node
  - Un i-node extra por cada enlace simbólico
  - Un enlace simbólico es un alias: un fichero varios nombres: ojo al recorrer el sistema de ficheros

# Implementación del sistema de ficheros. (gestión de espacio en disco) (16)

¿Cómo almacenar en disco los  $n$  bytes de un fichero?

- $N$  bytes contiguos en disco.
  - ¿y si el fichero crece?
- En bloques (contiguos o no) de tamaño fijo

## ¿Tamaño del bloque?

- cilindro
- pista
- sector

Si muy grande (cilindro): desperdicio  
tamaño medio de un fichero: 1K  
1 bloque: 32k  
se desperdicia el 97% del espacio

# Implementación del sistema de ficheros. (gestión de espacio en disco) (17)

Si muy pequeño: muchos accesos: muy lento. Ejem:

pista:  $2^{17}$  bytes = 131072 bytes

tiempo de rotación: 8,33 ms

tiempo medio de posicionamiento: 10 ms

Para leer un bloque de k bytes:

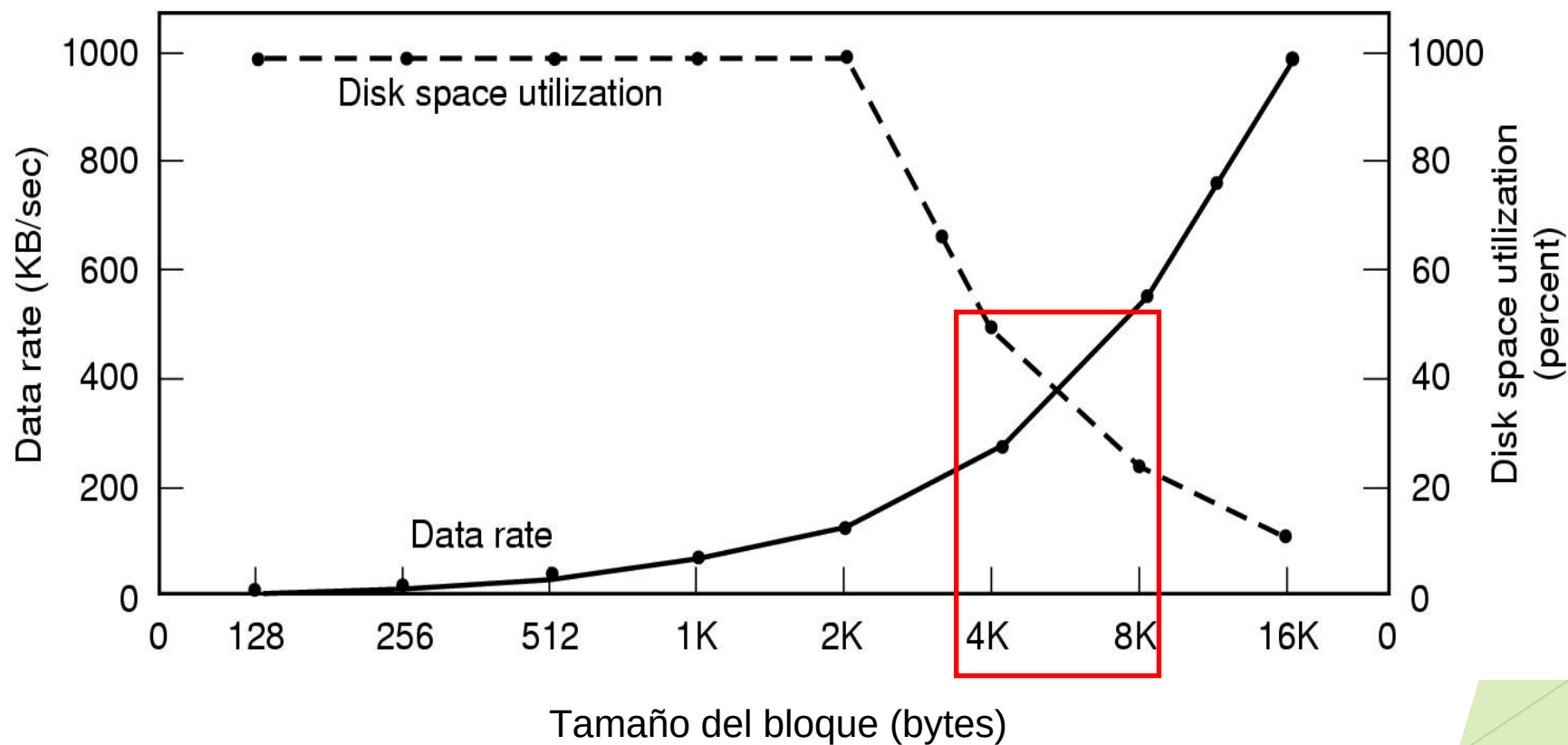
$T_{pos.} = 10$  ms

$T_{rot.} = 8,33/2 = 4,165$  ms

$T_{tra.} = (k/131072) * 8,33$

Si el bloque se muy pequeño, prima los tiempos de posicionamiento y de espera de rotación.

# Implementación del sistema de ficheros. (gestión de espacio en disco) (18)

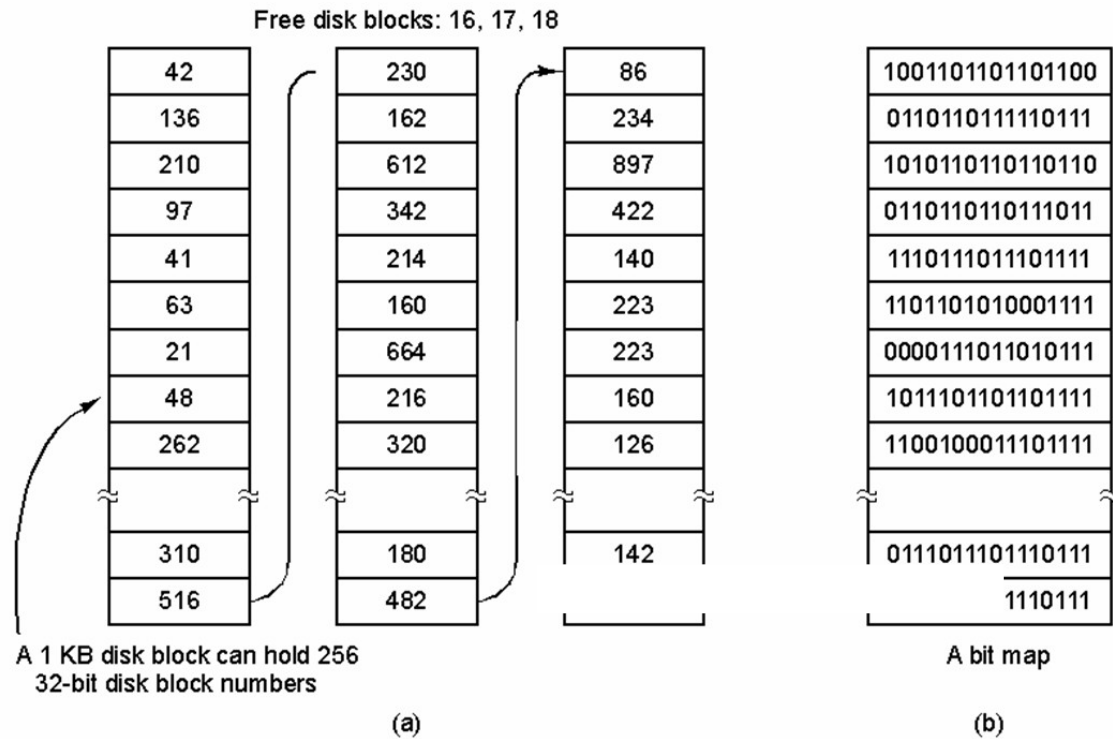


Todos los ficheros son de 2k

# Implementación del sistema de ficheros. (gestión de espacio en disco) (19)

**¿Cuáles son los bloques libres?**

(a) Lista enlazada de bloques libres; (b) mapa de bits



# Implementación del sistema de ficheros. (gestión de espacio en disco) (20)

## ¿Cuáles son los bloques libres?

Disco de 16Gb, tamaño del bloque: 1k

Número de bloques:  $2^4 \cdot 2^{30} / 2^{10} = 2^{24}$

32 bits para direccionar un bloque (¿por qué no 24?)

¿Cuánto ocupa cada método si disco vacío?

- Lista enlazada.
  - 1kb/4bytes = 255 dir. por bloque (+ 1 puntero)
  - $2^{24} / 255 = 65793$  bloques de 1kb

(b) Mapa de bits

- $2^{24} \text{ bits} / (2^{10} \cdot 2^3) = 2^{11} (2048)$  bloques de 1k

El epígrafe Disk Quotas (Cuotas de disco) (pág. 415) no entra para el examen

# Implementación del sistema de ficheros. (fiabilidad del sistema de ficheros) (24)

## Consistencia del sistema de ficheros

Por fallos hw o soft fallan lecturas/escrituras de bloques críticos:  
El sistema de ficheros queda en un estado inconsistente.

Programas que validan la consistencia del sistema de ficheros:

- fsck (Unix)
- scandisk (Windows)

Dos tipos de operaciones:

- consistencia de bloques
- consistencia de ficheros

¿Cómo se valida la consistencia de bloques?